



The Complete

Database Management & SQL Course

From First Principles to Production Queries

A complete, self-paced curriculum — 7 modules, hands-on SQL labs, normalization, relational algebra, transactions, and capstone projects.

2026 Edition

How to Use This Course

Read this page first — it tells you the fastest path to genuine SQL fluency.

What you will be able to do

By the end of this course you will be able to design a normalized relational schema from a real-world problem, write correct and efficient SQL for querying and modifying data, reason about relational algebra and query optimization, and understand how a database keeps your data safe and consistent under concurrent access and failures. You will have built three portfolio-ready projects along the way.

The structure of every module

- **Concept** — the idea explained in plain language, with diagrams and analogies.
- **Syntax & examples** — runnable SQL you can copy into any database.
- **Try it** — short challenges that force you to write code, not just read it.
- **Pitfalls** — the mistakes that trip up 90% of learners, called out early.
- **Check yourself** — review questions with answers at the end of each module.

A proven learning path

Research on how people actually become fluent in SQL (LearnSQL, Scaler, Coursera, and roadmap.sh learning paths, 2026) converges on one rule: **you learn by writing queries, not by reading about them.** Read a section, then immediately re-create the example yourself in a live database before moving on. Aim for the schedule below.

Phase	Modules	Focus	Suggested time
1 · Foundations	1 – 2	What a database is; data models, types, constraints	Weeks 1–2
2 · Design	3 + Normalization	ER diagrams, schema, 1NF–BCNF, DML	Weeks 3–4
3 · Querying	4 + Advanced SQL	Relational algebra, joins, subqueries, windows	Weeks 5–7
4 · Reliability	5	Transactions, concurrency, recovery, security	Week 8
5 · Capstone	Projects	Build & query three real schemas	Ongoing

SET UP A PLAYGROUND FIRST

Install a free database before Module 1: **SQLite** (zero setup, perfect for learning), **PostgreSQL** (industry standard, free), or **MySQL**. Or use a browser sandbox such as db-fiddle, SQLite Online, or programiz — no install needed. Every example in this course runs on standard SQL with notes where dialects differ.

Course Contents

MODULE 1 · Introduction to Databases, Transactions & Tables

What a database system is · purpose & functions of a DBMS
Tables, fields, records · primary, composite & foreign keys
Relational databases · views of data · DBMS architecture (1/2/3-tier)
Three-level architecture · transactions & ACID · first SQL

MODULE 2 · Data Models, Data Types & Constraints

Five data models · building blocks · business rules
Evolution of data models · degrees of data abstraction
SQL data types in depth · the seven constraint types

MODULE 3 · Database Design, ER Diagrams & Data Manipulation

ER diagrams · entities, attributes, relationships, cardinality
Weak vs strong entities · ER design issues · relational schema
ALTER / DROP / TRUNCATE · INSERT / UPDATE / DELETE · SELECT & WHERE

BONUS · Normalization & Functional Dependencies

Why normalize · anomalies · functional dependencies
1NF → 2NF → 3NF → BCNF with worked examples · when to denormalize

MODULE 4 · Relational Algebra

Selection, projection, set operations, rename
Joins · aggregate functions · GROUP BY / HAVING · comparison operators
Polyglot persistence · query optimization

BONUS · Advanced & Practical SQL

All join types visualized · subqueries & CTEs · window functions
CASE, set operators, performance habits that scale

MODULE 5 · Transaction Management & Concurrency Control

ACID in depth · serializability · 2PL · deadlocks (wait-die / wound-wait)
Timestamp & optimistic methods · recovery (WAL, checkpoints) · security

CAPSTONE · Projects, SQL Cheat-Sheet & Next Steps

Three build-and-query projects · one-page SQL reference · where to go next

Introduction to Databases, Transactions & Table Management

Where data lives, why a DBMS exists, and the vocabulary every section after this depends on.

1.1 What is a database system?

A **database** is an organized, structured collection of data — usually stored as tables of rows and columns. A **Database Management System (DBMS)** is the software that creates, maintains, secures and serves that data, giving users and applications a controlled way to store and retrieve it. Think of the database as a library's collection of books, and the DBMS as the entire library system: the catalogue, the librarians, the lending rules and the security.

A complete database system is made of several cooperating parts:

- **DBMS** — the core engine that handles creation, maintenance and access.
- **Database** — the structured data itself, organized into tables (entities).
- **Query language** — how you talk to the data; almost always **SQL** (Structured Query Language).
- **Database engine** — the low-level component for storage, query processing and transactions.
- **Data model** — the rules for how data is structured (relational, NoSQL, object-oriented).
- **Metadata** — “data about the data”: structure, constraints, indexes.
- **Users** — people or applications that query, update and administer the data.

1.2 The purpose & functions of a DBMS

Before databases, data lived in scattered files that were duplicated, inconsistent and hard to query. A DBMS solves this. Its ten core jobs are worth memorizing because every later topic is one of these jobs done in detail:

#	Function	What it gives you
1	Storage & management	Structured storage with enforced data integrity
2	Data retrieval	Fast, expressive querying via SQL
3	Data manipulation	Insert, update, delete — grouped into transactions
4	Security & authorization	Access control and encryption
5	Backup & recovery	Restore to a consistent state after failure
6	Concurrency control	Many users at once without corrupting data
7	Data integrity	Primary/foreign keys, unique & check constraints
8	Data abstraction	A logical view independent of physical storage
9	Performance optimization	Query optimization & resource management
10	Data independence	Change storage or logical structure without breaking apps

KEY IDEA — DATA INDEPENDENCE

Physical data independence lets you change how data is stored on disk without touching the logical schema or applications. **Logical data independence** lets you change the logical schema (add a column, split a table) with minimal impact on applications. This separation is the single most important reason databases scale across decades of changing requirements.

1.3 Tables, fields and records

The relational model gives us three nested ideas. Master this vocabulary now — the whole course uses it constantly.

- **Table (relation)** — a collection of related data in rows and columns; represents one entity, e.g. *Students*.
- **Field (column / attribute)** — one property of the entity, with a single data type, e.g. *Name* or *Age*.
- **Record (row / tuple)** — one instance of the entity: all field values for a single student.

Example — a Students table

StudentID (key)	Name	Age	Department
1	Alice	20	Computer Science
2	Bob	22	Electrical Engineering
3	Charlie	21	Mathematics

Each column is a field; each row is a record; StudentID uniquely identifies a row.

1.4 Keys: how rows are identified and linked

Primary key

A **primary key** is the column (or combination of columns) that uniquely identifies every row in a table. It guarantees that no two rows are the same and gives other tables a reliable way to point at a record.

- **Unique** — no two rows share the same key value.
- **Non-null** — every row must have a key value.
- **Immutable** — the value should never change once set.
- **Single or composite** — it can be one column or several together.
- **Auto-indexed** — the DBMS builds an index on it for fast lookups.

Composite primary key

When no single column is unique, combine columns. In a *StudentCourses* enrolment table, neither StudentID nor CourseID is unique alone — a student takes many courses and a course has many students — but the pair (**StudentID, CourseID**) is unique.

StudentID	CourseID	EnrollmentDate
101	CSE101	2025-01-01
102	CSE101	2025-01-02
101	CSE102	2025-01-03

Composite primary key = (StudentID, CourseID).

Foreign key

A **foreign key** is a column in one table that references the primary key of another, creating a relationship between them. It enforces **referential integrity**: the value must either match an existing key in the referenced table, or be NULL. This is how a relational database connects *Employees* to their *Departments*, or *Students* to their *Courses*.

MENTAL MODEL

Primary key = a row's **passport number** (unique, identifies it). Foreign key = a **reference to someone else's passport** (must point to a real one). Referential integrity is the rule that you can't reference a passport that doesn't exist.



1.5 Relational databases in action

A **relational database** organizes data into tables linked by keys. Two related tables — *Students* and *Courses* — show how a foreign key wires them together so we can answer questions that span both.

Students (CourseID is a foreign key)

StudentID	Name	Age	CourseID
1	Alice	20	101
2	Bob	22	102
3	Charlie	21	101

Courses (CourseID is the primary key)

CourseID	CourseName	Duration
101	Data Structures	3 months
102	DBMS	4 months

To list each student with their course name, we **join** the tables on the shared CourseID:

```
SELECT Students.Name, Courses.CourseName
FROM Students
JOIN Courses
ON Students.CourseID = Courses.CourseID;
```

Result

Name	CourseName
Alice	Data Structures
Bob	DBMS
Charlie	Data Structures

TRY IT

Re-create both tables in your playground with CREATE TABLE, INSERT the rows, then run the JOIN above. Next, modify it to show only students older than 20. (Hint: add a WHERE clause on Students.Age.)

1.6 The view of data: abstraction, schema & instance

A DBMS deliberately hides complexity. **Data abstraction** means users see a simple view while messy storage details stay hidden. Two terms make this precise:

- **Schema** — the *design* of the database (the structure of its tables). It rarely changes. There are three kinds: physical, logical and view schema.
- **Instance** — the actual *data* stored at a given moment. It changes constantly as rows are inserted, updated and deleted.

ANALOGY

The schema is a blank form (the fields and their types). An instance is the form filled in with one person's answers at one moment. The form design stays put; the answers change all day.

1.7 DBMS architecture: 1-tier, 2-tier, 3-tier

Architecture describes how users connect to the database. There are three common shapes:

Tier	Who talks to whom	Best for
1-tier	User sits directly on the DBMS; changes hit the database immediately.	Local development & quick experiments
2-tier	A client application talks straight to the database server via APIs like ODBC/JDBC.	Classic client-server desktop apps
3-tier	Client → application server → database. The client never touches the DB directly.	Large web applications (security & scale)

WHY 3-TIER WINS FOR THE WEB

The middle application-server layer adds security (the database is never exposed to the client), scalability (many app servers can share one database), and maintainability (business logic lives in one place). This is the architecture behind virtually every website you use.



1.8 The three-level architecture (ANSI-SPARC)

Independently of tiers, every database is described at three *levels of abstraction*. This model is the formal reason data independence works.

Level	Also called	What it describes	Maintained by
External	View level	What each user sees — only the slice they need	Application designers
Conceptual	Logical level	The whole logical design: tables, relationships, constraints	DBA
Internal	Physical level	How data is actually stored on disk; space allocation	DBA / engine

Multiple users can have different **external views** of the same conceptual schema; the conceptual schema maps down to the internal schema. Change the internal level (e.g. add an index) and the conceptual and external levels are untouched — that is physical data independence in action.

1.9 Transactions and the ACID properties

A **transaction** is a sequence of one or more operations treated as a single logical unit of work — it either fully happens or not at all. The classic example is a bank transfer: debit one account, credit another. To stay reliable, every transaction obeys four guarantees abbreviated **ACID**:

Property	Guarantee	Bank-transfer example
Atomicity	All operations succeed, or none do.	A half-finished transfer never leaves the system.
Consistency	The database moves from one valid state to another.	Total money across accounts is unchanged.
Isolation	Concurrent transactions don't interfere.	Two simultaneous transfers don't corrupt balances.
Durability	Once committed, changes survive crashes.	After a confirmed transfer, a crash can't undo it.

Transaction states

A transaction moves through a small lifecycle: **Active** (running) → **Partially committed** (operations done, not yet saved) → **Committed** (saved permanently). If something goes wrong it becomes **Failed** and is then **Aborted** (rolled back, all changes undone).

A transaction in SQL

```
BEGIN TRANSACTION;  
UPDATE Accounts SET Balance = Balance - 500 WHERE AccountID = 'A';  
UPDATE Accounts SET Balance = Balance + 500 WHERE AccountID = 'B';  
COMMIT;  
-- If any step fails, undo everything:  
ROLLBACK;
```

PITFALL

Forgetting to wrap multi-step changes in a transaction is the classic cause of corrupted data. If the debit succeeds but the credit fails and they weren't in one transaction, money simply vanishes. We return to transactions in depth in Module 5.



1.10 Your first SQL: create, populate, drop

Let's build a small library database end-to-end. Type each block into your playground in order. This is the first time you'll feel SQL working.

Step 1 — create the database

```
CREATE DATABASE LibraryDB;
```

Step 2 — create related tables (note the foreign keys)

```
CREATE TABLE Authors (  
  AuthorID INT PRIMARY KEY AUTO_INCREMENT,  
  FirstName VARCHAR(50),  
  LastName VARCHAR(50)  
);  
  
CREATE TABLE Books (  
  BookID INT PRIMARY KEY AUTO_INCREMENT,  
  Title VARCHAR(100),  
  AuthorID INT,  
  ISBN VARCHAR(13),  
  PublishedYear INT,  
  FOREIGN KEY (AuthorID) REFERENCES Authors(AuthorID)  
);  
  
CREATE TABLE Loans (  
  LoanID INT PRIMARY KEY AUTO_INCREMENT,  
  BookID INT,  
  BorrowerName VARCHAR(100),  
  BorrowDate DATE,  
  ReturnDate DATE,  
  FOREIGN KEY (BookID) REFERENCES Books(BookID)  
);
```

Step 3 — insert sample data

```
INSERT INTO Authors (FirstName, LastName)  
VALUES ('George', 'Orwell'), ('Aldous', 'Huxley');  
  
INSERT INTO Books (Title, AuthorID, ISBN, PublishedYear)  
VALUES ('1984', 1, '9780451524935', 1949),  
('Brave New World', 2, '9780060850524', 1932);  
  
INSERT INTO Loans (BookID, BorrowerName, BorrowDate, ReturnDate)  
VALUES (1, 'John Doe', '2024-01-15', '2024-01-22'),  
(2, 'Jane Smith', '2024-01-18', '2024-01-25');
```

To remove an entire database permanently (be careful — this is irreversible):

```
DROP DATABASE LibraryDB;
```

TRY IT — EXTEND THE SCHEMA

Add a *Members* table with MemberID, Name and JoinDate, then change Loans so BorrowerName becomes a MemberID foreign key referencing Members. You've just improved the design — exactly the kind of thinking Module 3 and the Normalization bonus formalize.

Check yourself — Module 1

Q. What is the difference between a database and a database system?

A. A database is the structured data itself; a database system (DBMS + database + users + tools) is the whole software environment that manages it.

Q. What three properties must a primary key have?

A. Unique, non-null, and (ideally) immutable.

Q. What does a foreign key enforce?

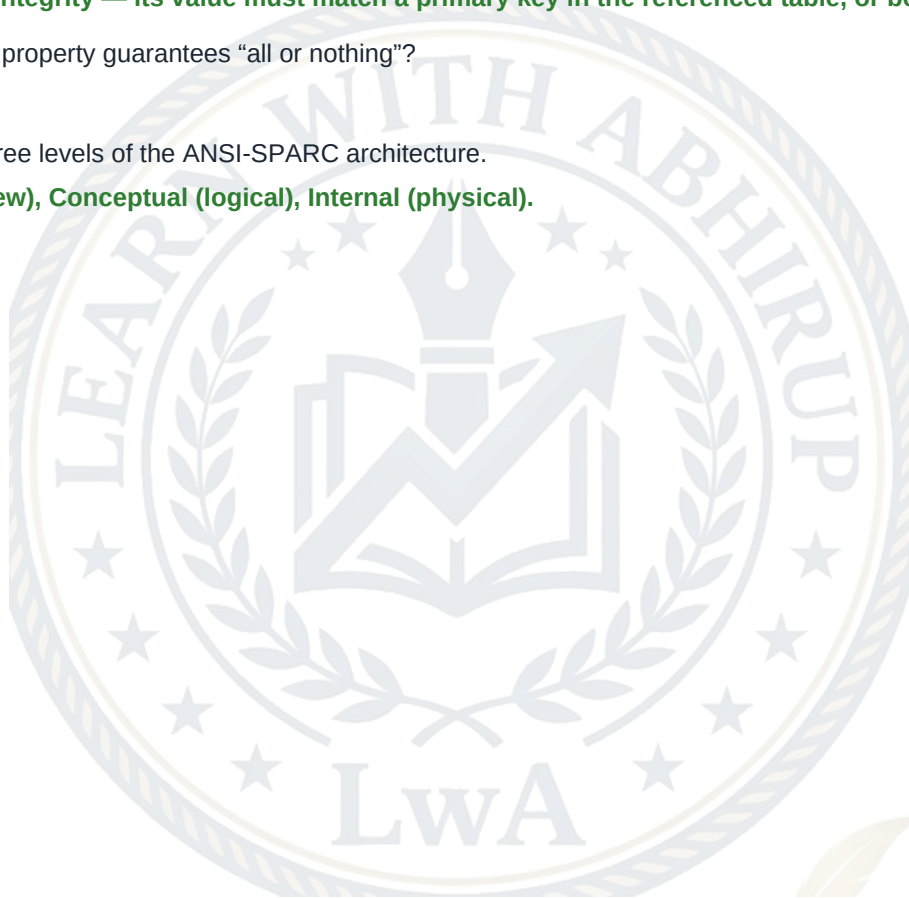
A. Referential integrity — its value must match a primary key in the referenced table, or be NULL.

Q. Which ACID property guarantees “all or nothing”?

A. Atomicity.

Q. Name the three levels of the ANSI-SPARC architecture.

A. External (view), Conceptual (logical), Internal (physical).



Data Models, Data Types & Constraints

The blueprints behind a database, the types your columns can hold, and the rules that keep data honest.

2.1 What is a data model?

A **data model** is an abstract representation of how data is structured, organized and related. It is the blueprint you design before building a single table — it decides what is stored, how pieces relate, and what rules apply. A good model gives consistency, integrity, clear communication between team members, and efficient retrieval.

2.2 The five data models

Model	How it organizes data	Good for
Hierarchical	Tree: each record has one parent, many children	Org charts, file systems
Network	Graph: nodes and edges, many-to-many links	Complex interconnected data
Relational	Tables (relations) of rows and columns, linked by keys	The default — simple & flexible
Entity-Relationship	Entities, attributes & relationships (a design tool)	Conceptual design phase
Object-Oriented	Objects bundling data + behavior (methods)	Complex data with logic

Beyond these, **NoSQL** models — document, key-value, wide-column and graph — handle large volumes of unstructured or semi-structured data with flexible schemas. The relational model remains the workhorse and the focus of this course because SQL is built on it.

2.3 The building blocks of a data model

Every data model is assembled from four elements. You'll see these again as the components of an ER diagram in Module 3:

- Entities** — real-world objects we store data about (Student, Employee). Drawn as rectangles.
- Attributes** — the properties of an entity (name, age, roll number). Drawn as ellipses.
- Relationships** — associations between entities (a Student *enrolls in* a Course). Drawn as diamonds. Three kinds: one-to-one, one-to-many, many-to-many.
- Constraints** — conditions applied to data to keep it valid (e.g. a borrower may hold at most 2 books).

2.4 Business rules

Business rules are statements that define or constrain some aspect of the business — they translate real-world policy into database rules. They range from simple (“a customer's age must be a number”) to complex (“a customer's credit limit cannot exceed three times their average monthly purchase”). DBMSs enforce them with several mechanisms:

- Constraints** — rules in the schema itself (NOT NULL, UNIQUE, CHECK).

- **Triggers** — actions that fire automatically in response to data changes.
- **Stored procedures** — reusable blocks of logic invoked by applications.
- **Check constraints** — verify validity before data is inserted or updated.

2.5 The evolution of data models

Understanding how we got to relational databases explains why they're designed the way they are.

- **Hierarchical & network (1960s–70s)** — efficient but rigid; complex relationships were hard to manage.
- **Relational (1970)** — E. F. Codd's breakthrough: data as simple tables with a strong mathematical foundation. SQL became the standard language. This is still the dominant model.
- **Object-oriented & object-relational** — added rich data types, inheritance and encapsulation for complex domains.
- **NoSQL & graph (2000s–)** — born from internet-scale data; trade some ACID guarantees for flexibility and horizontal scalability. Graph databases excel at highly connected data (social networks, fraud detection, recommendations).

THE DRIVERS OF CHANGE

Every new model answered the same four pressures: growing **data complexity**, higher **performance** demands, the need for **scalability**, and the desire for **flexibility** as business needs shift.

2.6 Degrees of data abstraction

The same three-level idea from Module 1 reappears here as the **three levels of data abstraction** — worth restating because it underpins good design:

Level	Hides	Concerned with	Typical user
Physical (lowest)	Nothing — it's the storage detail	Disk blocks, indexes, access methods	DBA / engine
Logical (conceptual)	Physical storage	What data exists and how it relates	Designers
View (highest)	The whole database's complexity	What each user needs to see	End users

2.7 Creating a table from another table

SQL lets you build a new table from an existing one — handy for backups, archives and filtered snapshots.

Same structure, no data

```
CREATE TABLE EmployeesBackup AS
SELECT * FROM Employees
WHERE 1 = 0; -- always false, so 0 rows copied (structure only)
```

Same structure and all data

```
CREATE TABLE EmployeesArchive AS
SELECT * FROM Employees;
```

Selected columns and filtered rows

```
CREATE TABLE ActiveEmployees AS
SELECT EmployeeID, FirstName, LastName
FROM Employees
WHERE Status = 'Active';
```

WHY WHERE 1 = 0 WORKS

The condition `1 = 0` is never true, so the `SELECT` returns no rows — but the engine still copies the column definitions. You get an empty table with an identical structure. A common exam question and a genuinely useful trick.

2.8 SQL data types

A **data type** defines what kind of value a column can hold. Choosing well prevents bugs, saves space and speeds queries. Names vary slightly across MySQL, PostgreSQL, SQL Server and Oracle, but the categories are universal.

Numeric

Type	Stores
INT / INTEGER	Whole numbers, no decimals
SMALLINT / BIGINT / TINYINT	Smaller / larger / very small integer ranges
FLOAT	Single-precision floating point
DOUBLE / DOUBLE PRECISION	Double-precision floating point
DECIMAL(p,s) / NUMERIC(p,s)	Exact fixed-point: p = total digits, s = digits after the point

Character & text

Type	Stores
CHAR(n)	Fixed-length string of exactly n characters
VARCHAR(n)	Variable-length string up to n characters
TEXT / CLOB	Large text, often with no fixed length limit

Date/time, boolean, binary & spatial

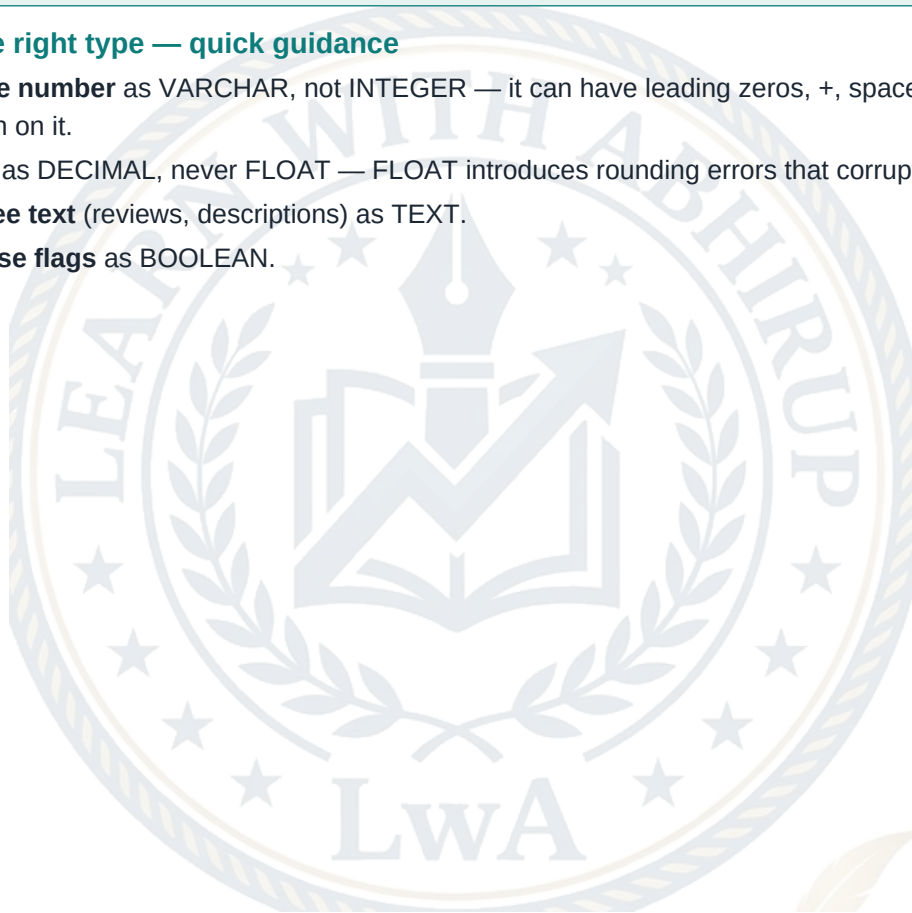
Type	Stores
DATE / TIME / DATETIME / TIMESTAMP	Dates, times, or both
TIMESTAMPPTZ	Date & time with time zone
BOOLEAN	TRUE / FALSE (sometimes NULL)
BINARY(n) / VARBINARY(n) / BLOB	Fixed/variable binary data; large objects (images, video)
GEOMETRY / POINT / POLYGON	Spatial / geometric data

FIXED vs VARIABLE LENGTH — A FAVORITE EXAM TOPIC

CHAR(n) always reserves n characters, padding with spaces — fast and ideal for codes of a known fixed size (country codes, fixed IDs). **VARCHAR(n)** stores only the characters you use plus a tiny length marker — ideal for names, emails and anything of varying length. Rule of thumb: use CHAR only when the length truly never varies; otherwise use VARCHAR.

Choosing the right type — quick guidance

- Store a **phone number** as VARCHAR, not INTEGER — it can have leading zeros, +, spaces, and you never do math on it.
- Store **money** as DECIMAL, never FLOAT — FLOAT introduces rounding errors that corrupt totals.
- Store **long free text** (reviews, descriptions) as TEXT.
- Store **true/false flags** as BOOLEAN.



2.9 Constraints: the rules that keep data honest

Constraints are rules applied to columns to enforce data integrity. They are your first and cheapest line of defense against bad data — the database itself rejects anything that breaks a rule. There are seven you must know.

The seven constraint types

Constraint	Ensures	Notes
PRIMARY KEY	Unique + non-null row identity	One per table; may be composite
FOREIGN KEY	Value matches a key in another table	Enforces referential integrity
UNIQUE	No duplicate values in a column	A table may have many; allows one NULL
NOT NULL	Column must always have a value	Makes a field mandatory
CHECK	Value satisfies a condition	e.g. Stock >= 0, GPA BETWEEN 0 AND 4
DEFAULT	A fallback value if none given	e.g. Status defaults to 'Pending'
INDEX	(Not a true constraint) speeds lookups	Trades write speed for read speed

All seven in one table

```
CREATE TABLE Employees (  
  EmployeeID INT PRIMARY KEY, -- unique identity  
  FirstName VARCHAR(50) NOT NULL, -- mandatory  
  LastName VARCHAR(50) NOT NULL,  
  Email VARCHAR(100) UNIQUE, -- no duplicates  
  Salary DECIMAL(10,2) CHECK (Salary > 0), -- must be positive  
  DepartmentID INT,  
  HireDate DATE DEFAULT CURRENT_DATE, -- fallback value  
  FOREIGN KEY (DepartmentID) REFERENCES Departments(DepartmentID)  
);
```

PRIMARY KEY vs UNIQUE — KNOW THE DIFFERENCE

Both forbid duplicates. The differences: a table has exactly **one** primary key but can have **many** unique constraints; a primary key **cannot** be NULL, while a UNIQUE column generally **can** hold one NULL. Use PRIMARY KEY for identity, UNIQUE for “this must also be distinct” (like an email).

2.10 Practice — design tables from a spec

Read each requirement, then check your CREATE TABLE against the answer. This is exactly how schema design questions appear on exams and in interviews.

Build a Students table:

- StudentID — integer, primary key
- FirstName, LastName — up to 50 chars, not null
- DateOfBirth — date, not null
- Email — up to 100 chars, unique
- GPA — decimal(3,2), must be between 0.00 and 4.00

```
CREATE TABLE Students (  
  StudentID INT PRIMARY KEY,  
  FirstName VARCHAR(50) NOT NULL,  
  LastName VARCHAR(50) NOT NULL,  
  DateOfBirth DATE NOT NULL,  
  Email VARCHAR(100) UNIQUE,  
  GPA DECIMAL(3,2) CHECK (GPA BETWEEN 0.00 AND 4.00)  
);
```

TRY IT

Now design a **Products** table yourself: ProductID (PK), ProductName (required, ≤100), Category (≤50), Price (decimal(8,2) > 0), StockQuantity (int ≥ 0), and SupplierID that must match a Suppliers table. Compare with the combined-constraints pattern above.

Check yourself — Module 2

Q. Name the five classic data models.

A. Hierarchical, Network, Relational, Entity-Relationship, Object-Oriented.

Q. What are the four building blocks of a data model?

A. Entities, Attributes, Relationships, Constraints.

Q. Which data type should store money, and why not FLOAT?

A. DECIMAL/NUMERIC — FLOAT introduces rounding errors that corrupt financial totals.

Q. What is the difference between CHAR and VARCHAR?

A. CHAR is fixed-length (padded); VARCHAR is variable-length (stores only what's used).

Q. How does PRIMARY KEY differ from UNIQUE?

A. One PK per table, never NULL; many UNIQUE constraints allowed, each may hold one NULL.

Database Design, ER Diagrams & Data Manipulation

Turn a real-world problem into a diagram, the diagram into tables, and then change the data inside them.

3.1 The Entity-Relationship (ER) diagram

An **ER diagram** is a visual blueprint of a database — you draw it *before* creating tables. It captures the entities you store, their attributes, and the relationships between them. Designing on paper first prevents expensive mistakes later. An ER model has three main components: **entities, attributes, and relationships**.

The visual language of ER diagrams

Shape	Means
Rectangle	Entity set
Double rectangle	Weak entity set
Ellipse	Attribute
Underlined ellipse	Key attribute (uniquely identifies the entity)
Double ellipse	Multivalued attribute (e.g. multiple phone numbers)
Dashed ellipse	Derived attribute (e.g. age, computed from date of birth)
Diamond	Relationship set
Double diamond	Identifying relationship (links a weak entity)
Lines	Connect attributes to entities, entities to relationships
Double lines	Total participation in a relationship

3.2 Entities and attributes

Strong vs weak entities

A **strong entity** has its own primary key and stands on its own (e.g. *Student*). A **weak entity** cannot be uniquely identified by its own attributes — it depends on a strong entity. Classic example: a bank *Account* can't be identified without knowing its *Bank*, so *Account* is weak. A weak entity uses a **partial key** combined with the strong entity's key.

Strong entity set	Weak entity set
Has its own primary key	Lacks enough attributes for a primary key
Drawn as a rectangle	Drawn as a double rectangle
Key shown by underline	Partial key shown by a dashed underline
Called the dominant entity	Called the subordinate entity
Identified by its own attribute	Identified by strong key + partial key

The four attribute types

- **Key attribute** — uniquely identifies an entity (student roll number). Underlined.
- **Composite attribute** — made of sub-parts (Address = street + city + pin code).
- **Multivalued attribute** — can hold several values (a person's phone numbers). Double ellipse.
- **Derived attribute** — computed from another (Age from Date of Birth). Dashed ellipse.

3.3 Relationships and cardinality

Cardinality describes how many instances of one entity relate to another. There are four cardinal relationship types — recognising them is essential for both ER design and choosing foreign keys:

Type	Meaning	Example
One-to-One (1:1)	One instance ↔ one instance	A person has one passport
One-to-Many (1:N)	One instance ↔ many instances	A customer places many orders
Many-to-One (N:1)	Many instances ↔ one instance	Many students study at one college
Many-to-Many (M:N)	Many ↔ many	Students assigned to many projects, and vice-versa

DESIGN TIP — RESOLVING MANY-TO-MANY

A relational database can't store a many-to-many relationship directly. You resolve it with a **junction (bridge) table** holding two foreign keys — exactly the StudentCourses table from Module 1. Spotting an M:N relationship and creating its junction table is one of the most important design skills you'll build.

3.4 ER design issues to avoid

- 1 **Entity set vs attribute** — don't store another entity's primary key as a plain attribute; model it as a relationship instead.
- 2 **Entity set vs relationship set** — use a relationship to describe an *action between* entities; don't fold it into an entity.
- 3 **Binary vs n-ary relationships** — most relationships are binary; a complex (ternary) relationship can often be split into several binary ones.
- 4 **Placing relationship attributes** — attach an attribute where its cardinality dictates; for many-to-many relationships, attributes usually belong on the relationship itself.

3.5 From ER diagram to relational schema

A **relation schema** defines the structure of a table: its name, attributes, each attribute's domain (type), and its constraints. Translating an ER diagram into schemas is the bridge from design to implementation. The components of a relation schema are the relation name, attribute names, domains, the primary key, any foreign keys, and constraints.

A university domain decomposes into clear schemas (primary keys in bold context, foreign keys link tables):

```
student (rollNo, name, degree, year, sex, deptNo, advisor)
department (deptId, name, hod, phone)
course (courseId, name, credits, deptNo)
professor (empId, name, sex, startYear, deptNo, phone)
enrollment (rollNo, courseId, sem, year, grade)
teaching (empId, courseId, sem, year, classroom)
prerequisite(preReqCourse, courseId)
```

The links are foreign keys: *student.deptNo* → *department.deptId*; *student.advisor* → *professor.empId*; *enrollment.rollNo* → *student.rollNo*; and so on. Every relationship in the ER diagram becomes either a foreign key or a junction table.

3.6 Modifying schema: ALTER, DROP, TRUNCATE

ALTER TABLE — change an existing table's structure

```
ALTER TABLE Students ADD Email VARCHAR(255); -- add a column
ALTER TABLE Students MODIFY COLUMN Email VARCHAR(100); -- change a type
ALTER TABLE Students DROP COLUMN Email; -- remove a column
ALTER TABLE Students RENAME COLUMN old_name TO new_name; -- rename a column
ALTER TABLE Students RENAME TO Learners; -- rename the table
```

DROP vs TRUNCATE vs DELETE — a crucial distinction

Command	Removes	Keeps table structure?	Can use WHERE?
DROP TABLE	The entire table (and its data)	No — table is gone	No
TRUNCATE TABLE	All rows, fast	Yes — empty table remains	No
DELETE	Rows you choose	Yes	Yes

PITFALL — THESE ARE NOT INTERCHANGEABLE

DROP destroys the table itself. **TRUNCATE** empties it instantly but is hard to undo and ignores most safety checks. **DELETE** is the only one that can target specific rows with a WHERE clause and can be rolled back inside a transaction. Reach for DELETE by default; use TRUNCATE only when you truly want every row gone.

3.7 Manipulating data: INSERT, UPDATE, DELETE, SELECT

INSERT — add new rows

```
INSERT INTO Users VALUES (1, 'John Doe', 'john@example.com'); -- all columns
INSERT INTO Users (name, email) VALUES ('Jane Doe', 'jane@example.com'); -- some columns
```

UPDATE — modify existing rows (always with WHERE!)

```
UPDATE Users  
SET email = 'newemail@example.com'  
WHERE id = 1;
```

DELETE — remove rows

```
DELETE FROM Users WHERE id = 1; -- one row  
DELETE FROM Users; -- ALL rows (structure stays)
```

THE MOST EXPENSIVE MISTAKE IN SQL

Running **UPDATE** or **DELETE** without a **WHERE** clause changes or erases every row in the table. Always write the WHERE clause first, run a SELECT with the same WHERE to preview which rows you'll hit, and only then change UPDATE/DELETE. Professionals do this every single time.



3.8 SELECT and the WHERE clause

SELECT retrieves data — it's the command you'll use most. **WHERE** filters which rows come back.

```
SELECT * FROM Users; -- every column, every row
SELECT name, email FROM Users; -- specific columns
SELECT * FROM Users WHERE name = 'John Doe'; -- filtered rows
```

Combining conditions with AND / OR / NOT

```
SELECT * FROM Users WHERE name = 'John Doe' AND email = 'john@example.com';
SELECT * FROM Users WHERE name = 'John Doe' OR name = 'Jane Doe';
SELECT * FROM Users WHERE NOT name = 'John Doe';
```

SUMMARY — THE DML TOOLKIT

INSERT adds rows, **UPDATE** changes them, **DELETE** removes them, and **SELECT** reads them — with **WHERE** filtering and **AND/OR/NOT** combining conditions. These five verbs cover the vast majority of everyday database work.

Check yourself — Module 3

Q. What shape represents an entity, a relationship, and an attribute in an ER diagram?

A. Rectangle = entity, Diamond = relationship, Ellipse = attribute.

Q. How is a weak entity drawn, and why is it 'weak'?

A. Double rectangle. It can't be uniquely identified by its own attributes; it depends on a strong entity.

Q. How do you implement a many-to-many relationship in a relational database?

A. With a junction (bridge) table containing two foreign keys.

Q. Which removes data but keeps the table structure: DROP, TRUNCATE, or both?

A. TRUNCATE (and DELETE). DROP removes the table entirely.

Q. What happens if you run UPDATE without a WHERE clause?

A. Every row in the table is updated — a common and costly accident.

Normalization & Functional Dependencies

The discipline that turns a messy table into a clean, anomaly-free schema — the missing half of database design.

Why normalize?

Normalization is a step-by-step process of organizing data into smaller, related tables to **reduce redundancy** and **preserve integrity**. Storing the same fact in many places causes three classic **anomalies**:

- **Insertion anomaly** — you can't add a fact because unrelated data is missing (can't add a course with no enrolled student if course data lives in the enrolment table).
- **Update anomaly** — a fact stored in many rows must be changed in all of them; miss one and the data becomes inconsistent.
- **Deletion anomaly** — deleting one row accidentally erases an unrelated fact.

REAL-WORLD COST

When Healthcare.gov launched in 2013 it nearly collapsed on day one. Post-incident audits found schemas violating 1NF and 2NF, causing massive redundancy and update anomalies that spiked server load. Remediation reportedly cost around \$300 million. Normalization isn't academic — it's a prerequisite for reliable, scalable systems.

Functional dependencies — the foundation

A **functional dependency** $X \rightarrow Y$ means: if you know X, you know exactly one Y. We say “X determines Y.” For example, $StudentID \rightarrow StudentName$: a student ID determines exactly one name. Normalization is essentially the art of arranging tables so that every attribute depends on “the key, the whole key, and nothing but the key.”

The normal forms, step by step

Form	Requirement	Removes
1NF	Atomic values; no repeating groups; unique rows	Multi-valued cells / lists
2NF	1NF + every non-key attribute depends on the WHOLE key	Partial dependencies
3NF	2NF + no non-key attribute depends on another non-key	Transitive dependencies
BCNF	3NF + every determinant is a candidate key	Overlapping-candidate-key anomalies

THE GOLDEN RULE OF 3NF

Every non-key column must depend on **the key, the whole key, and nothing but the key**. “The key” $\rightarrow$ 1NF needs a key. “The whole key” $\rightarrow$ 2NF (no partial dependency). “nothing but the key” $\rightarrow$ 3NF (no transitive dependency).

1NF — atomic values

A table is in **First Normal Form** if every cell holds a single, indivisible value and each row is unique. The violation: storing a list in one cell.

Violates 1NF (multiple phone numbers in one cell)

StudentID	Name	PhoneNumbers
1	Ali	98765, 91234

In 1NF (one value per cell, one row per phone)

StudentID	Name	PhoneNumber
1	Ali	98765
1	Ali	91234

2NF — no partial dependencies

A table is in **Second Normal Form** if it is in 1NF *and* every non-key attribute depends on the **entire** composite key, not just part of it. Partial dependency only arises with composite keys.

Consider an enrolment table with composite key (**StudentID, CourseID**) and attributes StudentName and Instructor. Here *StudentName depends only on StudentID* and *Instructor depends only on CourseID* — both partial dependencies. Split them out:

```
Students (StudentID, StudentName)
Courses (CourseID, Instructor)
Enrollment (StudentID, CourseID) -- the composite key, no partial deps
```

3NF — no transitive dependencies

A table is in **Third Normal Form** if it is in 2NF *and* no non-key attribute depends on another non-key attribute. If $StudentID \rightarrow DeptID$ and $DeptID \rightarrow DeptName$, then *DeptName depends on StudentID only through DeptID* — a transitive dependency. Move department details into their own table:

```
Students (StudentID, Name, DeptID)
Departments (DeptID, DeptName) -- DeptName now lives with its key
```

BCNF — every determinant is a key

Boyce-Codd Normal Form (informally “3.5NF”) is a stricter 3NF for the rare case of overlapping candidate keys. The rule: for every functional dependency $X \rightarrow Y$, *X must be a **superkey***. A table can satisfy 3NF yet still have a non-key attribute determining part of a key; BCNF decomposition fixes that by splitting until every left-hand side of a dependency is a candidate key.

When NOT to normalize: denormalization

Normalization optimizes for **writes and integrity**, but heavy normalization means more joins, which can slow **reads**. **Denormalization** deliberately reintroduces some redundancy to speed up read-heavy analytics. This is why data warehouses use **star schemas** for fast reporting. The professional rule of thumb: **normalize to 3NF for transactional systems (OLTP); denormalize selectively for analytical systems (OLAP)** where

reads dominate.

PRACTICAL TARGET

Most production databases are normalized to **3NF** — it removes nearly all redundancy while keeping joins manageable. Reach for BCNF when 3NF still leaves anomalies, and denormalize only with a measured performance reason.

Check yourself — Normalization

Q. What three anomalies does normalization prevent?

A. Insertion, update, and deletion anomalies.

Q. State the golden rule of 3NF.

A. Every non-key column depends on the key, the whole key, and nothing but the key.

Q. Which normal form removes partial dependencies? Which removes transitive ones?

A. 2NF removes partial dependencies; 3NF removes transitive dependencies.

Q. When would you intentionally denormalize?

A. For read-heavy analytical (OLAP) workloads where joins slow down reporting — e.g. a star schema.



Relational Algebra

The mathematics underneath SQL: a small set of operations that combine and filter relations to answer any question.

4.1 What relational algebra is

Relational algebra is a procedural query language that operates on relations (tables) and produces new relations. It is the theoretical foundation of SQL — every SELECT you write maps to a combination of these operators. Three terms recur: a **relation** is a table, a **tuple** is a row, and an **attribute** is a column.

4.2 Selection (σ) and Projection (π)

Selection (σ) filters *rows* by a condition. **Projection** (π) picks *columns*. Together they're the algebraic core of SELECT ... WHERE.

Operation	Algebra	SQL equivalent
Select salary > 60000	$\sigma(\text{Salary} > 60000)(\text{Employee})$	SELECT * FROM Employee WHERE Salary > 60000
Project Name & Salary	$\pi(\text{Name}, \text{Salary})(\text{Employee})$	SELECT Name, Salary FROM Employee

REMEMBER

σ (**sigma**) = **rows** (the WHERE filter). π (**pi**) = **columns** (the SELECT list). Mixing these up is the most common relational-algebra exam slip.

4.3 Set operations

Because relations are sets of tuples, classic set operations apply (the two relations must be **union-compatible** — same columns):

- **Union** ($\cup$) — all tuples from both relations, duplicates removed.
- **Set Difference** ($-$) — tuples in the first relation but not the second.
- **Intersection** ($\cap$) — tuples present in both relations.
- **Cartesian Product** ($\times$) — every combination of tuples from both (rarely used alone).

Worked example — Set Difference finds 'what's missing'

To find courses no student has enrolled in, subtract the enrolled course IDs from all courses:

```
NotTaken = Courses -  $\pi(\text{CourseID})(\text{EnrolledCourses})$ 
```

CourseID	CourseName
C104	Operating Systems

Given courses C101–C104 and enrolments in only C101, C102, C103, the difference leaves C104.

4.4 Rename (ρ) and Join ($\bowtie$)

The **Rename** (ρ) operator changes a relation's or attribute's name — vital for self-joins (joining a table to itself), avoiding name clashes in unions, clarifying complex queries, and adapting to schema changes.

The **Natural Join** ($\bowtie$) combines two relations on their common attributes, automatically matching equal values and dropping duplicate columns. Joining Employee and Department on their shared DeptID gives each employee enriched with their department name — the algebraic root of the SQL JOIN you'll use constantly.

EmpID	Name	DeptID	DeptName
1	Alice	101	Engineering
2	Bob	102	Marketing

Result of Employee $\bowtie$ Department.



4.5 Aggregate functions & grouping

Aggregate functions compute a single value from many rows. The five essentials:

Function	Returns	Example
COUNT	Number of rows	COUNT(EmpID)
SUM	Total of values	SUM(Salary)
AVG	Average value	AVG(Salary)
MAX	Largest value	MAX(Amount)
MIN	Smallest value	MIN(Amount)

GROUP BY splits rows into groups so an aggregate is computed per group. Total sales by region:

```
SELECT Region, SUM(Amount) AS TotalSales
FROM Sales
GROUP BY Region;
```

HAVING vs WHERE — the distinction examiners love

Criterion	WHERE	HAVING
Filters	Individual rows	Groups (aggregated results)
With aggregates?	No (e.g. can't use SUM)	Yes — designed for them
Runs	Before grouping	After grouping & aggregation
Example	WHERE salary > 30000	HAVING COUNT(*) > 5

```
SELECT DeptID, AVG(Salary)
FROM Employee
GROUP BY DeptID
HAVING AVG(Salary) > 60000; -- filter the GROUPS, not the rows
```

ORDER OF EXECUTION (memorize this)

SQL logically runs in this order: **FROM** → **WHERE** → **GROUP BY** → **HAVING** → **SELECT** → **ORDER BY**. That's why WHERE can't see aggregates (they don't exist yet) but HAVING can (grouping is done). Understanding this order resolves most 'why won't my query work' confusion.

4.6 Relational comparison operators

Operator	Meaning	Example
=, !=, <>	Equal / not equal	WHERE Age = 30
>, <, >=, <=	Greater / less than (or equal)	WHERE Price > 100
BETWEEN	Within an inclusive range	WHERE Amount BETWEEN 500 AND 1000
IN / NOT IN	Matches (or not) a set	WHERE Dept IN ('HR','IT')
LIKE	Pattern match with wildcards	WHERE Name LIKE 'A%'

Operator	Meaning	Example
IS NULL / IS NOT NULL	Tests for missing values	WHERE Discount IS NULL

PITFALL — NULL IS NOT A VALUE

Never write `= NULL`. `NULL` means 'unknown,' and any comparison with it yields neither true nor false. Always use **IS NULL / IS NOT NULL**. This trips up nearly every beginner at least once.

4.7 Polyglot persistence & query optimization

Polyglot persistence means using different databases for different jobs within one application. A modern e-commerce platform might run PostgreSQL for orders (ACID transactions), MongoDB for product catalogs (flexible documents), Neo4j for recommendations (graph relationships), and InfluxDB for real-time metrics (time-series). Use the right tool for each data shape.

Query optimization is the DBMS choosing the most efficient way to run a query. Key strategies: **indexing** (a sorted lookup structure that avoids full-table scans), **query rewriting** (transforming a query into a cheaper but equivalent form), and **cost-based optimization** (estimating the cost of alternative plans and picking the cheapest). Indexing is the single biggest lever — it can turn a multi-second scan into a millisecond lookup.

Check yourself — Module 4

Q. Which operator filters rows, and which selects columns?

A. Selection (σ) filters rows; Projection (π) selects columns.

Q. What does Set Difference ($A - B$) return?

A. Tuples that are in A but not in B — great for finding 'what's missing.'

Q. WHERE vs HAVING: which one works on groups?

A. HAVING filters groups after aggregation; WHERE filters individual rows before grouping.

Q. State SQL's logical order of execution.

A. FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY.

Q. How should you test for a missing value?

A. With IS NULL / IS NOT NULL — never = NULL.

Advanced & Practical SQL

The skills that separate someone who 'knows SQL' from someone who is fluent: every join type, subqueries, CTEs, and window functions.

A.1 Every JOIN type, visualized

Joins combine rows from two tables on a related column. Learning all four types — and *when* each is right — is the milestone that unlocks real analytical SQL.

Join	Returns	Use when
INNER JOIN	Only rows that match in BOTH tables	You only want related records
LEFT JOIN	All rows from the left + matches from the right (NULLs if none)	You want everything from the main table, matched or not
RIGHT JOIN	All rows from the right + matches from the left	Same idea, reversed (rarely needed)
FULL OUTER JOIN	All rows from both, matched where possible	You want the complete picture from both sides

```
-- INNER: customers who have placed at least one order
SELECT c.name, o.order_id
FROM customers c
INNER JOIN orders o ON c.customer_id = o.customer_id;

-- LEFT: ALL customers, with their orders if any (NULL when none)
SELECT c.name, o.order_id
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id;
```

THE MENTAL MODEL FOR JOINS

Think of two overlapping circles. **INNER** = the overlap only. **LEFT** = the whole left circle (plus overlap). **RIGHT** = the whole right circle. **FULL** = both circles entirely. A LEFT JOIN where the right side is NULL is the standard way to find 'rows with no match' — e.g. customers who never ordered.

A.2 Subqueries — queries inside queries

A **subquery** is a query nested inside another. It lets you solve a problem in stages: compute something inner, then use it outer. Subqueries teach you to break complex requirements into manageable pieces — a core analytical skill.

```
-- Employees who earn more than the company average
SELECT name, salary
FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);

-- Customers who HAVE placed an order (using EXISTS)
SELECT name FROM customers c
WHERE EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id);
```

PERFORMANCE TIP

For existence checks on large tables, **EXISTS** usually outperforms **IN** with a subquery, because it stops as soon as it finds one match rather than building the full list.

A.3 Common Table Expressions (CTEs)

A **CTE** is a named, temporary result defined with **WITH**. It makes complex queries readable by naming each step — think of it as a variable for a query. CTEs are the modern, preferred alternative to deeply nested subqueries.

```
WITH dept_avg AS (
  SELECT department, AVG(salary) AS avg_sal
  FROM employees
  GROUP BY department
)
SELECT e.name, e.salary, d.avg_sal
FROM employees e
JOIN dept_avg d ON e.department = d.department
WHERE e.salary > d.avg_sal; -- earns above their dept average
```


A.4 Window functions — the analyst's superpower

A **window function** computes a value across a set of rows *related to the current row*, **without collapsing them**. This is the key difference from GROUP BY: GROUP BY reduces many rows to one summary row; a window function keeps every row and adds the computed value alongside it. They are the hallmark of intermediate-to-advanced SQL in 2026.

```
-- Rank employees by salary WITHIN each department, keeping all rows
SELECT name, department, salary,
RANK() OVER (PARTITION BY department ORDER BY salary DESC) AS dept_rank
FROM employees;

-- Running total of sales over time
SELECT sale_date, amount,
SUM(amount) OVER (ORDER BY sale_date) AS running_total
FROM sales;
```

Function	Does
ROW_NUMBER()	Sequential number per row in the window
RANK() / DENSE_RANK()	Ranking, with / without gaps after ties
SUM/AVG/COUNT() OVER(...)	Running or partitioned aggregates that keep all rows
LAG() / LEAD()	Value from the previous / next row (e.g. month-over-month change)

GROUP BY vs WINDOW — THE ONE-LINE DIFFERENCE

GROUP BY collapses rows into summaries (fewer rows out). A **window function** computes the same kind of metric but **keeps every original row**, attaching the result as a new column. Need a running total or a per-group ranking while still seeing each row? That's a window function.

A.5 CASE & set operators

CASE adds if/then logic inside a query; **UNION** stacks result sets vertically (UNION removes duplicates, UNION ALL keeps them and is faster).

```
SELECT name, salary,
CASE WHEN salary >= 80000 THEN 'Senior'
WHEN salary >= 50000 THEN 'Mid'
ELSE 'Junior' END AS band
FROM employees;

SELECT name FROM customers_us
UNION ALL -- keep duplicates; faster than UNION
SELECT name FROM customers_uk;
```

A.6 Performance habits that scale

- **Avoid SELECT *** in production — name only the columns you need; less data moved.
- **Index** the columns you filter and join on most; they turn scans into instant lookups.
- **Filter early** — apply WHERE before joins and aggregations to shrink the working set.

- **Don't wrap indexed columns in functions** — write `hire_date >= '2024-01-01'` instead of `YEAR(hire_date) = 2024`; the second prevents index use.
- **Prefer EXISTS over IN** for existence checks; **UNION ALL over UNION** when duplicates are fine.
- **Read the execution plan (EXPLAIN)** to see what the database actually does.

A REAL SPEEDUP

One documented case: rewriting `YEAR(hire_date) = 2024` as `hire_date >= '2024-01-01'` let the query use an index and ran **6× faster** — from 1m53s down to 20s. Small habits, large results.

Check yourself — Advanced SQL

Q. How do you find rows in table A with no match in table B?

A. LEFT JOIN A to B and keep rows WHERE B's key IS NULL (or use NOT EXISTS).

Q. What is the key difference between GROUP BY and a window function?

A. GROUP BY collapses rows into summaries; a window function keeps all rows and adds the computed value.

Q. What does WITH ... AS define?

A. A Common Table Expression — a named temporary result that makes complex queries readable.

Q. Why prefer `hire_date >= '2024-01-01'` over `YEAR(hire_date) = 2024`?

A. Wrapping an indexed column in a function prevents the index from being used; the range form keeps it.



Transaction Management & Concurrency Control

How a database stays correct when many users hit it at once and when the power suddenly goes out.

5.1 ACID, revisited and deepened

Module 1 introduced ACID; here we see how the database actually delivers each guarantee. A transaction is a single logical unit of work, and ACID is the contract that makes it trustworthy:

- **Atomicity** — all-or-nothing, enforced by the ability to **roll back** partial work.
- **Consistency** — every transaction moves the database between valid states; constraints and triggers enforce the rules.
- **Isolation** — concurrent transactions appear to run one-at-a-time; delivered by **concurrency control**.
- **Durability** — committed changes survive crashes; delivered by **transaction logs** (write-ahead logging).

5.2 Serializability & concurrency control

Serializability is the gold standard of isolation: the result of running transactions concurrently must equal the result of *some* serial (one-after-another) order. **Concurrency control** is the machinery that achieves it without making everything actually run one at a time (which would be far too slow).

Two-Phase Locking (2PL)

The most common protocol. Each transaction has two phases that guarantee serializability:

- **Growing phase** — the transaction acquires locks but releases none.
- **Shrinking phase** — the transaction releases locks but acquires no new ones.

Locks come in two flavors: a **shared lock** (many transactions may read together) and an **exclusive lock** (one transaction writes, blocking all others).

5.3 Deadlocks and how to handle them

A **deadlock** is when two transactions each hold a lock the other needs, so both wait forever. Databases resolve this with timestamp-based prevention schemes:

Scheme	Rule	Effect
Wait-Die	Older waits for younger; younger requesting an older's lock is aborted ('dies') and restarts	Non-preemptive — only younger transactions die
Wound-Wait	Older preempts ('wounds') younger by forcing it to roll back; younger waits for older	Preemptive — older transactions win

HOW WAIT-DIE WORKS, STEP BY STEP

Every transaction gets a timestamp at start (older = smaller). If an **older** transaction wants a resource held by a **younger** one, it **waits**. If a **younger** one wants a resource held by an **older** one, it **aborts and restarts** with the same timestamp. Either way there's no circular wait — so no deadlock.

Deadlocks can also be **detected** after the fact using a **wait-for graph**: if the graph contains a cycle, a deadlock exists and one transaction is rolled back to break it.

5.4 Timestamp & optimistic methods

Timestamp ordering assigns each transaction a unique timestamp and orders conflicts by it, tracking a **read timestamp (RTS)** and **write timestamp (WTS)** per data item; operations that would violate the timestamp order are rejected.

Optimistic concurrency control assumes conflicts are rare and uses **no locks** during work. It runs in three phases — **Read** (work freely), **Validation** (check whether anyone else touched your data), **Write** (commit if valid, else roll back). Ideal for **low-contention** systems; locking (pessimistic) is better when conflicts are frequent.



5.5 Database recovery management

Recovery restores the database to a consistent state after a failure. Failures come in three kinds: **transaction failure** (logical error, deadlock), **system failure** (crash, power loss), and **media failure** (physical disk damage).

Write-Ahead Logging (WAL)

The cornerstone technique. Its rule is in the name: **write the change to a log *before* writing it to the database**. That log lets the system, after a crash, decide what to redo and what to undo:

- **Redo** — re-apply changes from *committed* transactions whose writes hadn't reached disk (preserves Durability).
- **Undo** — reverse changes from *incomplete* transactions (preserves Atomicity).

Checkpointing periodically flushes a consistent snapshot to disk, so recovery only needs to replay the log since the last checkpoint — dramatically shortening restart time.

WHY WAL GUARANTEES DURABILITY & ATOMICITY

Because the log is written first, a crash mid-transaction still leaves a complete record of intent. On restart, the database replays committed work (redo) and rolls back unfinished work (undo) — so you never lose a confirmed transaction and never keep a half-finished one.

5.6 Distributed transactions & message queues

When a transaction spans multiple systems, the **Two-Phase Commit (2PC)** protocol ensures all participants either commit together or roll back together — a **prepare** phase (everyone votes) followed by a **commit** phase. A **message queue** supports this by passing messages between components asynchronously, providing **decoupling** (parts work independently) and **fault tolerance** (if a component fails, messages wait safely in the queue rather than being lost).

5.7 Database security

Protecting data from unauthorized access and misuse rests on five pillars:

Pillar	Protects by
Authentication	Verifying <i>who</i> is connecting
Authorization	Controlling <i>what</i> each user may access (often via Role-Based Access Control)
Encryption	Scrambling data at rest and in transit (e.g. Transparent Data Encryption)
Auditing	Tracking access and activity to detect misuse
Access controls	Enforcing rules on who may read or modify data

DEFENSE IN DEPTH

Authentication, authorization, and encryption work as layers: authentication confirms identity, authorization limits reach, and encryption ensures that even stolen data is unreadable. No single layer is enough — real security stacks them.

Check yourself — Module 5

Q. What do the growing and shrinking phases of 2PL do?

A. Growing: acquire locks (release none). Shrinking: release locks (acquire none). Together they guarantee serializability.

Q. In Wait-Die, what happens when a younger transaction wants an older one's lock?

A. The younger transaction aborts ('dies') and restarts with the same timestamp.

Q. State the rule of Write-Ahead Logging.

A. Write the change to the log before writing it to the database.

Q. Which recovery actions preserve Durability and Atomicity?

A. Redo (re-apply committed changes) preserves Durability; Undo (reverse incomplete ones) preserves Atomicity.

Q. Name three pillars of database security.

A. Any three of: authentication, authorization, encryption, auditing, access controls.



Projects, Cheat-Sheet & Next Steps

Knowledge becomes skill only when you build. Three projects of rising difficulty, a one-page reference, and a path forward.

Three build-and-query projects

Do these in order in your playground. Each pulls together a cluster of skills from the course, mirroring the project-based roadmaps recommended for 2026 learners.

Project 1 — Movie database (basic querying)

- **Build:** tables for Movies (id, title, year, genre, rating) and Directors (id, name), linked by a foreign key.
- **Practice:** CREATE TABLE, INSERT, SELECT with WHERE, ORDER BY, and the comparison operators.
- **Stretch:** list the top 5 highest-rated movies per genre.

Project 2 — Employee dashboard (joins & aggregation)

- **Build:** Employees, Departments, and Salaries tables.
- **Practice:** INNER and LEFT JOIN, GROUP BY, HAVING, COUNT/SUM/AVG.
- **Stretch:** use a window function to rank employees by salary within each department.

Project 3 — Inventory & orders system (design + integrity)

- **Build:** Products, Suppliers, Customers, Orders, and an OrderItems junction table for the many-to-many between orders and products.
- **Practice:** normalize to 3NF, enforce constraints, write multi-table joins and subqueries/CTEs, and wrap a stock-decrement-plus-order-insert in a transaction.
- **Stretch:** a running monthly-revenue report using window functions; add indexes and compare query plans.

HOW TO PRACTICE SO IT STICKS

Read a section, then immediately re-create its example yourself in a live database before moving on. Write the query from memory; only peek when stuck. Reproduction — not re-reading — is what converts understanding into fluency.

One-Page SQL Cheat-Sheet

Tear this out (figuratively). Everything you reach for most, in one place.

Define & change structure (DDL)

```
CREATE DATABASE name; DROP DATABASE name;
CREATE TABLE t (col TYPE constraints, ...);
ALTER TABLE t ADD col TYPE; ALTER TABLE t DROP COLUMN col;
ALTER TABLE t MODIFY COLUMN col TYPE; ALTER TABLE t RENAME TO new;
DROP TABLE t; TRUNCATE TABLE t; -- empties; keeps structure
```

Change data (DML)

```
INSERT INTO t (c1, c2) VALUES (v1, v2);
UPDATE t SET c1 = v1 WHERE condition; -- ALWAYS use WHERE
DELETE FROM t WHERE condition; -- ALWAYS use WHERE
```

Query data (DQL)

```
SELECT c1, c2 FROM t
WHERE condition -- filter rows
GROUP BY c1 -- form groups
HAVING aggregate_condition -- filter groups
ORDER BY c1 DESC -- sort
LIMIT 10; -- cap rows
-- Logical order: FROM > WHERE > GROUP BY > HAVING > SELECT > ORDER BY
```

Joins

```
SELECT * FROM a INNER JOIN b ON a.id = b.a_id; -- matches in both
SELECT * FROM a LEFT JOIN b ON a.id = b.a_id; -- all of a + matches
SELECT * FROM a FULL OUTER JOIN b ON a.id = b.a_id;
```

Aggregates, subqueries, CTEs, windows

```
COUNT(*) SUM(c) AVG(c) MIN(c) MAX(c)
WHERE c IN (SELECT ...) -- subquery
WITH cte AS (SELECT ...) SELECT * FROM cte; -- CTE
RANK() OVER (PARTITION BY g ORDER BY c DESC) -- window function
```

Operators & transactions

```
= != > < >= <= BETWEEN a AND b IN (...) LIKE 'A%'
IS NULL IS NOT NULL AND OR NOT
BEGIN TRANSACTION; ... COMMIT; -- or ROLLBACK;
```

Where to Go Next

Skills to layer on top of this course

- **Stored procedures, triggers & views** — push logic into the database itself.
- **Indexing strategy & EXPLAIN plans** — go from 'queries work' to 'queries are fast.'
- **Cloud data warehouses** — BigQuery, Snowflake, Databricks all speak SQL at massive scale.
- **BI & visualization** — connect SQL to Power BI or Tableau to turn queries into dashboards.
- **NoSQL where it fits** — document, key-value and graph stores for non-relational shapes.

Career paths this opens

SQL remains one of the most stable, in-demand skills in tech — it has barely changed in decades while frameworks come and go, and it underpins analytics, business intelligence, backend development and data engineering alike. Roles include **Data Analyst**, **Business Intelligence Analyst**, **Backend Developer**, **Database Administrator**, and **Data Engineer** — the last of which commonly sits in the \$100k–\$120k range as of 2026.

THE ONE HABIT THAT MATTERS MOST

Keep a live database open whenever you study. Every concept in this course — from a CHECK constraint to a window function — takes thirty seconds to test for real. The people who become fluent are simply the ones who ran the queries themselves. You now have the map; go write SQL.

Built on a structured DBMS & SQL syllabus and enriched with normalization, advanced SQL, and modern best practices.